Guion de trabajo tutorizado

Workshop

Plataformas Tecnológicas

Profesores D. José Ignacio Merino y D. José Antonio Lagares
Máster Big Data & Business Analytics (MBDA)





Alumno: ÁLVARO RAMOS BERNABÉ

ENUNCIADO DE LA ACTIVIDAD

Guion oficial: «MBDA - Actividad - Workshop.pdf» (Campus Virtual UPO).

Objetivos

• Establecer un contacto más profundo con el mundo Big Data.

• Conocer algunas herramientas/empresas/tecnologías del sector.

• Repasar los conceptos vistos en la asignatura

Ejercicios ML

E1. 1 punto (60 minutos) Desarrolla un algoritmo de ML con el dataset para calcular el retraso de un vuelo.

• Existe total libertad sobre el problema a abordar.

• Puedes usar notebooks (Anaconda, Colab, Jupyter, etc.) o herramienta Cloud.

• No se permite usar herramienta SaaS para hacer todo el procesamiento sin tener el control del algoritmo que se usa.

• El algoritmo elegido no puede ser una red neuronal.

E2. 1 punto (60 minutos) Creado el algoritmo anterior, realice una red neuronal densa y compárela con el algoritmo anteriormente usado usando la matriz de confusión o métricas pertinentes (AUC, Curvas ROC, Medida F, etc.). La red deberá orientarse a calcular el retraso de un vuelo.

E3. 1 punto (60 minutos) Una vez creado todos los algoritmos anteriores, compare entrenar y evaluar los algoritmos usando la técnica “Cross Validation” frente a modelos de evaluación “80-20” y “70-30”.

E4. 1 Punto (60 minutos) Realice una hiperparametrización de uno de los modelos. Puede hacerlo de forma manual, o usando librerías como scikitlearn o alguna propia de frameworks (Hadoop, Spark u otros).

• Una vez realice la hiperparametrización, compruebe, evaluando el modelo de nuevo, si existe sobreajuste. ¿Cómo puede evitarlo?

Ejercicios Arquitecturas Cloud

E5. 6 puntos (300 minutos) Realice una arquitectura Cloud que permita:

• Ingestar datos.

• Procesar datos.

• Aplicar algoritmos de ML/IA.

• Guardar resultados.

• Visualizar resultados. Ponderaciones según el proveedor Cloud elegido:

• Hasta 4 puntos: Usando proveedor Cloud Azure o Google Cloud

• Hasta 6 puntos: Usando AWS, IBM u otros.

Fecha y Modo de entrega

Para la entrega de los ejercicios deberá subirse a la plataforma virtual en un único PDF.

La fecha de entrega está marcada dentro del Aula Virtual.

Servicios usados y facturación Cloud Por cada ejercicio implicado en levantar elementos Cloud, se deben adjuntar evidencias de que han eliminado todos los servicios usados. Adjunte capturas de pantalla u otro medio oportuno según el proveedor Cloud. En caso de dejar servicios levantados las consecuencias pueden afectar económicamente en caso de agotar el crédito gratuito o la capa gratuita

RESOLUCIÓN

Planteamiento y datos de partida

El conjunto de datos es `flights_subsampling.csv` (muestra del *On-Time Performance* del Departamento de Transporte de EE. UU., año 2015), junto con los catálogos `airlines.csv` y `airports.csv`. El problema elegido es de **clasificación binaria**: predecir si un vuelo llegará con retraso, entendido según el criterio de la FAA como una llegada con **más de 15 minutos** sobre la hora programada (`ARRIVAL_DELAY > 15`).

- Se descartan los vuelos cancelados (`CANCELLED = 1`) y desviados (`DIVERTED = 1`), porque no tienen hora de llegada real, y los registros sin `ARRIVAL_DELAY`. Quedan **89.675 vuelos**, de los que el **35,1 %** son retrasados.
- Solo se usan variables **conocidas antes del despegue**: mes, día del mes, día de la semana, hora programada de salida y de llegada, duración programada, distancia, aerolínea, aeropuerto de origen y de destino. Incluir `DEPARTURE_DELAY` o los campos `*_DELAY` de causas daría métricas artificialmente altas (fuga de información), porque solo se conocen una vez el vuelo ha ocurrido.
- Los aeropuertos se reducen a los **40 más frecuentes** y el resto se agrupa en la categoría `OTRO`, para evitar miles de columnas tras la codificación.
- El preprocesado es común a todos los modelos (`comun.py`): `StandardScaler` en las variables numéricas y `OneHotEncoder(handle_unknown='ignore')` en las categóricas, dentro de un `ColumnTransformer` encapsulado en un `Pipeline` de scikit-learn. Así la transformación se ajusta **solo con los datos de entrenamiento** en cada partición o *fold*, sin filtraciones.
- Partición base: *hold-out* 80-20 **estratificado** (`random_state=42`), manteniendo la proporción de retrasos en ambos conjuntos.

Toda la parte de ML (E1-E4) se ha ejecutado en local con Python 3 y scikit-learn; la parte de arquitectura (E5) se ha desplegado en Google Cloud. El código se entrega junto a este documento.

Ejercicio 1. Algoritmo de ML (sin redes neuronales)

Se entrenan dos algoritmos clásicos sobre el mismo `Pipeline`: una **regresión logística** como línea base interpretable y un **Random Forest** de 300 árboles con `class_weight='balanced_subsample'`, que captura interacciones no lineales entre hora, aerolínea y ruta.

```
aramber@localhost:~$ python3 e1_ml.py
Vuelos no cancelados ni desviados: 89675 | % retrasados (>15 min): 35.1

== LogisticRegression (baseline) ==  accuracy=0.6667  F1=0.5875  AUC=0.7316
Matriz de confusión [[TN FP],[FN TP]]: [[7702, 3937], [2040, 4256]]
              precision    recall  f1-score   support

     puntual      0.791     0.662     0.720     11639
   retrasado      0.519     0.676     0.587      6296

    accuracy                          0.667     17935
   macro avg      0.655     0.669     0.654     17935
weighted avg      0.695     0.667     0.674     17935


== RandomForestClassifier ==  accuracy=0.7144  F1=0.6239  AUC=0.7774
Matriz de confusión [[TN FP],[FN TP]]: [[8562, 3077], [2046, 4250]]
              precision    recall  f1-score   support

     puntual      0.807     0.736     0.770     11639
   retrasado      0.580     0.675     0.624      6296

    accuracy                          0.714     17935
   macro avg      0.694     0.705     0.697     17935
weighted avg      0.727     0.714     0.719     17935
```

Figura 1. Ejecución de `e1_ml.py`: métricas, matriz de confusión e informe de clasificación de ambos modelos.

Modelo	Accuracy	F1 (clase retrasado)	AUC
Regresión logística	0,6667	0,5875	0,7316
Random Forest (300 árboles)	0,7144	0,6239	**0,7774**

Lectura de los resultados. El Random Forest mejora la línea base en todas las métricas (+4,6 puntos de AUC). Es el comportamiento esperado: el retraso depende de combinaciones no lineales —por ejemplo, un vuelo de última hora de la tarde en un *hub* congestionado arrastra los retrasos acumulados durante el día—, y un modelo lineal no puede representarlas sin ingeniería de variables explícita. La matriz de confusión del Random Forest, `[[8562, 3077], [2046, 4250]]`, muestra un *recall* de 0,675 sobre los vuelos retrasados: detecta dos de cada tres, a costa de 3.077 falsas alarmas. Para un caso de uso operativo (avisar al pasajero o reasignar tripulaciones) ese equilibrio es razonable, porque **el coste de no avisar de un retraso es mayor que el de un aviso preventivo**; si se quisiera priorizar la precisión bastaría con subir el umbral de decisión por encima de 0,5.

El techo de AUC en torno a 0,78 es coherente con la información disponible: sin meteorología, estado de la aeronave ni retrasos encadenados del mismo avión, una parte importante del fenómeno es sencillamente impredecible.

Ejercicio 2. Red neuronal densa y comparación

Red densa `MLPClassifier` con tres capas ocultas (**128-64-32**), activación ReLU, optimizador Adam, `early_stopping=True` (10 % de validación interna, paciencia de 10 épocas) y `alpha=1e-4` como regularización L2. Se detuvo en la **época 18** con una *accuracy* de validación de 0,7480. Recibe exactamente el mismo preprocesado que el Random Forest, de modo que la comparación mide el algoritmo y no los datos.

```
aramber@localhost:~$ python3 e2_red.py
Red densa 3 capas ocultas (128-64-32, ReLU, Adam): 18 épocas, mejor val. accuracy=0.7480
RandomForest         matriz confusión [[TN FP],[FN TP]] = [[8562, 3077], [2046, 4250]]
Red neuronal densa   matriz confusión [[TN FP],[FN TP]] = [[8839, 2800], [2061, 4235]]
           modelo  accuracy  precision  recall     F1    AUC
     RandomForest    0.7144      0.580  0.6750 0.6239 0.7774
Red neuronal densa    0.7290      0.602  0.6726 0.6354 0.7907
```

Figura 2. Ejecución de `e2_red.py`: entrenamiento de la red densa y tabla comparativa con el Random Forest.

Modelo	Accuracy	Precisión	Recall	F1	AUC
Random Forest	0,7144	0,580	0,6750	0,6239	0,7774
Red neuronal densa 128-64-32	0,7290	0,602	0,6726	0,6354	**0,7907**

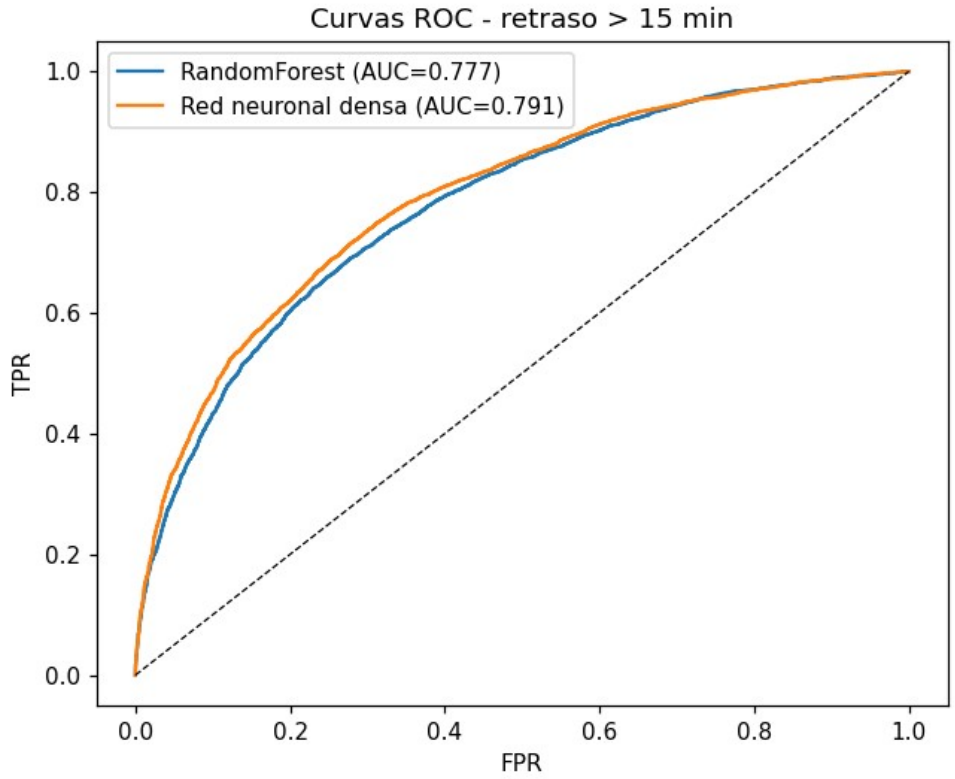


Figura 3. Curvas ROC de ambos modelos sobre el mismo conjunto de test (20 %).

Comparación. La red gana 1,3 puntos de AUC y 1,2 de F1. Las matrices de confusión explican en qué se diferencian: la red `[[8839, 2800], [2061, 4235]]` reduce en 277 los falsos positivos del Random Forest `[[8562, 3077], [2046, 4250]]` manteniendo prácticamente los mismos aciertos sobre retrasos, es decir, **acierta más sin volverse más alarmista**. Las curvas ROC se solapan en casi todo el recorrido y la red queda ligeramente por encima en la zona de bajo ratio de falsos positivos, que es la región útil si se quiere avisar solo de los vuelos con mayor riesgo.

Aun así, la diferencia es pequeña y tiene contrapartidas: la red necesita escalado obligatorio, es sensible a la inicialización y a la tasa de aprendizaje, tarda más en entrenar y no ofrece una importancia de variables directa. El Random Forest es más robusto y explicable. Con datos

tabulares de este tamaño, **ninguna de las dos familias domina claramente**: la elección depende tanto del coste de explotación como de la métrica.

Ejercicio 3. Validación cruzada frente a hold-out 80-20 y 70-30

Cada modelo se evalúa con tres esquemas: *hold-out* 80-20, *hold-out* 70-30 y **validación cruzada estratificada de 5 particiones** (`StratifiedKFold`, `shuffle=True`). En la validación cruzada el `Pipeline` completo se reajusta en cada *fold*, de modo que el escalado y la codificación nunca ven los datos de validación.

```
aramber@localhost:~$ python3 e3_validacion.py
       modelo        evaluación  accuracy     F1    AUC desv. AUC
 RandomForest    hold-out 80-20    0.7144 0.6239 0.7774         -
 RandomForest    hold-out 70-30    0.7114 0.6220 0.7749         -
 RandomForest CV 5-fold (media)    0.7155 0.6257 0.7778   ±0.0035
Red densa MLP    hold-out 80-20    0.7523 0.6008 0.7907         -
Red densa MLP    hold-out 70-30    0.7465 0.6028 0.7883         -
Red densa MLP CV 5-fold (media)    0.7489 0.6012 0.7896   ±0.0038
```

Figura 4. Ejecución de `e3_validacion.py`: comparación de los tres esquemas de evaluación.

Modelo	Evaluación	Accuracy	F1	AUC
Random Forest	hold-out 80-20	0,7144	0,6239	0,7774
Random Forest	hold-out 70-30	0,7114	0,6220	0,7749
Random Forest	CV 5-fold (media)	0,7155	0,6257	0,7778 ± 0,0035
Red densa MLP	hold-out 80-20	0,7523	0,6008	0,7907
Red densa MLP	hold-out 70-30	0,7465	0,6028	0,7883
Red densa MLP	CV 5-fold (media)	0,7489	0,6012	0,7896 ± 0,0038

Conclusiones. Las tres estimaciones coinciden dentro de ±0,003 de AUC, lo que indica que con casi 90.000 registros el *hold-out* ya es estable: el conjunto de test es lo bastante grande para que la varianza del muestreo sea pequeña. Aun así, la validación cruzada aporta dos cosas que el *hold-out* no puede dar:

- **Una medida de incertidumbre.** La desviación típica entre *folds* (±0,0035) permite afirmar que la diferencia de AUC entre la red y el Random Forest (0,013) es real y no ruido de partición; sin ese dato, comparar 0,7907 con 0,7774 sería un acto de fe.
- **Uso completo de los datos.** Cada observación se usa para entrenar en 4 de los 5 *folds* y para validar exactamente una vez. Con el 70-30 se desperdicia un 30 % de los datos en cada ajuste, y de hecho es el esquema que da sistemáticamente las métricas más bajas: menos datos de entrenamiento, modelo algo peor.
- **Coste.** La contrapartida es el tiempo: 5 *folds* suponen entrenar 5 veces. Por eso lo habitual es usar validación cruzada para **seleccionar** modelo e hiperparámetros y reservar un *hold-out* final intacto para la **estimación** del rendimiento, que es justo el procedimiento del ejercicio 4.
- En series temporales este esquema no sería válido: habría que usar `TimeSeriesSplit` para no entrenar con vuelos posteriores a los que se validan.

Ejercicio 4. Hiperparametrización y análisis de sobreajuste

Se optimiza el Random Forest con `RandomizedSearchCV` (12 combinaciones, validación cruzada de 3 particiones, métrica AUC) sobre la rejilla `n_estimators` ∈ {200, 400}, `max_depth` ∈ {10, 14, 18, 24, None}, `min_samples_leaf` ∈ {1, 3, 5, 10, 20} y `max_features` ∈ {sqrt, 0.3}. Con `return_train_score=True` se obtiene, para cada combinación, el AUC de entrenamiento y el de validación, cuya diferencia (**brecha**) es la medida directa del sobreajuste.

```
aramber@localhost:~$ python3 e4_hiper.py
max_depth  min_leaf max_feat  n_est  AUC_train_cv  AUC_val_cv  brecha
     None         3      0.3    400        0.9671      0.7836  0.1835
       18         1     sqrt    200        0.9006      0.7778  0.1227
     None        10     sqrt    200        0.8257      0.7778  0.0479
       24         1      0.3    400        0.9831      0.7769  0.2062
     None         1      0.3    400        1.0000      0.7726  0.2274
       24        20      0.3    200        0.8226      0.7721  0.0505
       14        10      0.3    400        0.8131      0.7697  0.0434
       14         3     sqrt    200        0.8067      0.7667  0.0400
       18        20     sqrt    400        0.7833      0.7635  0.0198
       14        10     sqrt    200        0.7835      0.7617  0.0218
       10         3      0.3    400        0.7890      0.7575  0.0315
       10         5      0.3    400        0.7856      0.7573  0.0283

Mejores hiperparámetros: {'n_estimators': 400, 'min_samples_leaf': 3, 'max_features': 0.3, 'max_depth': None}
Modelo óptimo -> AUC train=0.9649  AUC test=0.7920  brecha=0.1729
Árboles sin límite (max_depth=None, min_leaf=1) -> AUC train=1.0000  AUC test=0.7835  brecha=0.2165  <- sobreajuste
```

Figura 5. Ejecución de `e4_hiper.py`: las 12 combinaciones ordenadas por AUC de validación, modelo óptimo y comprobación del sobreajuste.

Mejor combinación: `n_estimators=400`, `min_samples_leaf=3`, `max_features=0.3`, `max_depth=None` → AUC de test **0,7920** (frente a 0,7774 del modelo por defecto). El modelo optimizado supera incluso a la red neuronal del ejercicio 2.

¿Existe sobreajuste? ¿Cómo evitarlo?

Sí, y la tabla lo cuantifica. El modelo óptimo obtiene AUC 0,9649 en entrenamiento frente a 0,7920 en test: una **brecha de 0,173**. Llevado al extremo, un bosque con árboles sin límite y `min_samples_leaf=1` alcanza un AUC de entrenamiento **perfecto (1,0000)** y baja a 0,7835 en test, con una brecha de 0,217: memoriza cada vuelo del entrenamiento sin generalizar mejor. En el otro extremo, `max_depth=18` con `min_samples_leaf=20` reduce la brecha a 0,0198 pero pierde AUC de validación (0,7635).

Conviene distinguir entre **brecha** y **rendimiento**: la configuración con menos sobreajuste no es la mejor, porque lo que importa es el error en datos no vistos. Medidas para controlarlo:

- **Limitar la complejidad de los árboles:** `max_depth`, `min_samples_leaf` o `min_samples_split` mayores, y `max_features` reducido, que además decorrela los árboles del bosque.
- **Aumentar el número de árboles.** En un Random Forest, subir `n_estimators` reduce la varianza del promedio sin aumentar el sobreajuste; no es un hiperparámetro que haya que "ajustar a la baja".
- **Evaluar siempre con validación cruzada** y con un conjunto de test que no intervenga en la búsqueda, como se ha hecho aquí: la búsqueda usa solo los datos de entrenamiento y el test se reserva para la estimación final.

- **Más datos y menos variables irrelevantes:** agrupar categorías poco frecuentes (los aeropuertos fuera del top-40) ya evita que el modelo aprenda ruido de rutas con dos o tres vuelos.
- **Alternativas con regularización explícita:** *gradient boosting* con `learning_rate` bajo y parada temprana, o penalizaciones L1/L2 en modelos lineales y redes.

Ejercicio 5. Arquitectura cloud de extremo a extremo (Google Cloud)

Se ha construido una arquitectura completa en **Google Cloud** sobre el proyecto `mbda-gvdm-2026` y la región `europe-west1`, que cubre las cinco etapas pedidas: ingesta, procesado, ML, almacenamiento de resultados y visualización. Para no depender de la muestra del enunciado, la ingesta descarga el fichero **real** de *On-Time Performance* del BTS (enero de 2015, 212 MB de CSV, 469.968 vuelos).

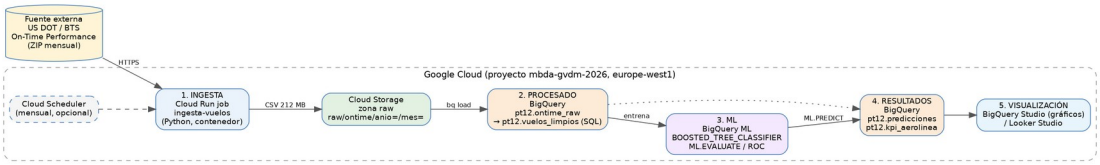


Figura 6. Arquitectura implementada en Google Cloud: Cloud Run Jobs (ingesta) → Cloud Storage (data lake) → BigQuery (procesado y BigQuery ML) → tablas de resultados → Looker Studio.

Etapa	Servicio	Implementación
Ingesta	Cloud Run Jobs + Artifact Registry	Contenedor Python que descarga el ZIP del BTS, lo descomprime y escribe el CSV en la zona `raw` del lago. Se despliega con `gcloud run jobs deploy --source` y puede programarse con Cloud Scheduler.
Lago de datos	Cloud Storage	Bucket `mbda-pt12-lago` con la ruta particionada `raw/ontime/anio=2015/mes=01/`, almacenamiento barato y desacoplado del cómputo.
Procesado	BigQuery	Carga con autodetección a `ontime_raw` (469.968 filas) y consulta SQL que genera la zona curada `vuelos_limpios` (457.013 filas, 20,22 % de retrasos).
ML/IA	BigQuery ML	Modelo `BOOSTED_TREE_CLASSIFIER` entrenado con SQL, sin mover los datos ni gestionar infraestructura.
Resultados	BigQuery	Tablas `predicciones` (probabilidad por vuelo) y `kpi_aerolinea` (agregado por compañía) como capa de servicio.
Visualización	Looker Studio	Cuadro de mando conectado directamente a `kpi_aerolinea`.

Por qué esta combinación. Todos los servicios son *serverless* y de pago por uso: no hay clúster que dimensionar ni que apagar. Separar el lago (Cloud Storage) del almacén analítico (BigQuery) permite reprocesar el histórico sin volver a descargarlo, y entrenar el modelo dentro de BigQuery evita exportar millones de filas a una máquina de entrenamiento. El patrón es el clásico de tres zonas: **bronce** (`raw`, dato tal cual llega), **plata** (`vuelos_limpios`, dato curado) y **oro** (`kpi_aerolinea`, dato agregado para consumo).

5.1. Ingesta

El *job* se empaqueta en un contenedor (`Dockerfile` de tres líneas sobre `python:3.12-slim`) y se despliega desde el código fuente; Cloud Build genera la imagen y la publica en

Artifact Registry. El destino en el lago está particionado por año y mes, de forma que una ejecución mensual programada con Cloud Scheduler iría acumulando el histórico sin sobrescribir nada.

```
gcloud run jobs deploy ingesta-vuelos --source . --region europe-west1 \
  --set-env-vars BUCKET=$LAGO,ANIO=2015,MES=1 --tasks 1 --max-retries 1 --task-
timeout 1800
gcloud run jobs execute ingesta-vuelos --region europe-west1 --wait
```



Figura 7. Ejecución del job `ingesta-vuelos`: 212,9 MB descargados del BTS y escritos en `gs://mbda-pt12-lago/raw/ontime/anio=2015/mes=01/`.

5.2. Procesado

El CSV se carga en BigQuery con autodetección de esquema y la zona curada se construye con una única consulta SQL, que filtra cancelados y desviados y deriva las variables previas al despegue (día de la semana, hora programada de salida, duración, distancia) más la etiqueta `retraso`.

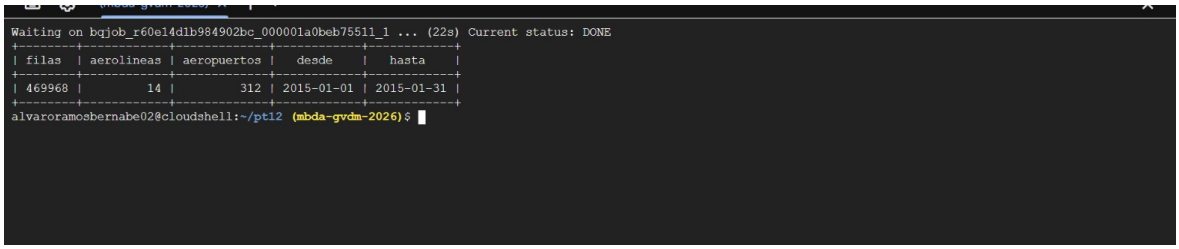


Figura 8. Carga de la tabla `pt12.ontime_raw` (469.968 filas) desde el lago.

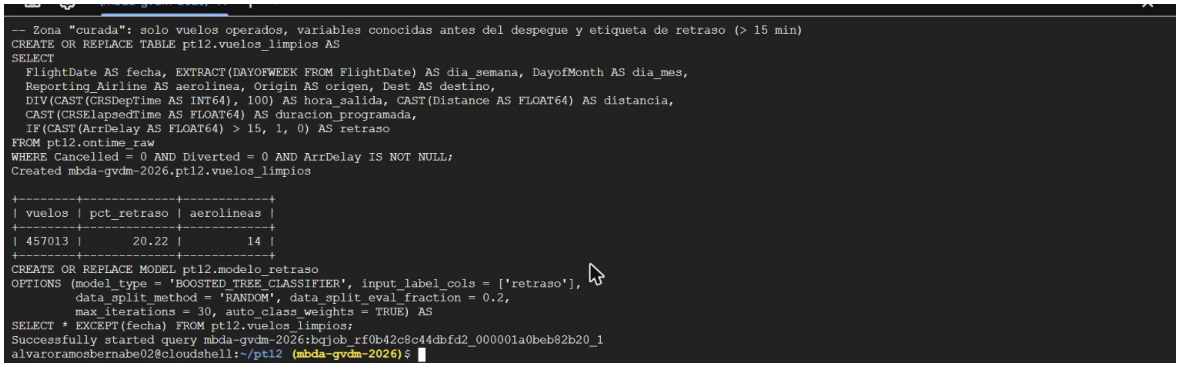


Figura 9. Creación de `pt12.vuelos_limpios`: 457.013 vuelos, de los que el 20,22 % llegan con más de 15 minutos de retraso.

5.3. Algoritmo de ML con BigQuery ML

El modelo se entrena con SQL sobre la tabla curada, con partición aleatoria del 20 % para evaluación y `auto_class_weights = TRUE` para compensar el desequilibrio entre clases (20 % de retrasos).

```
CREATE OR REPLACE MODEL pt12.modelo_retraso
OPTIONS (model_type = 'BOOSTED_TREE_CLASSIFIER', input_label_cols = ['retraso'],
         data_split_method = 'RANDOM', data_split_eval_fraction = 0.2,
         max_iterations = 30, auto_class_weights = TRUE) AS
SELECT * EXCEPT(fecha) FROM pt12.vuelos_limpios;
```

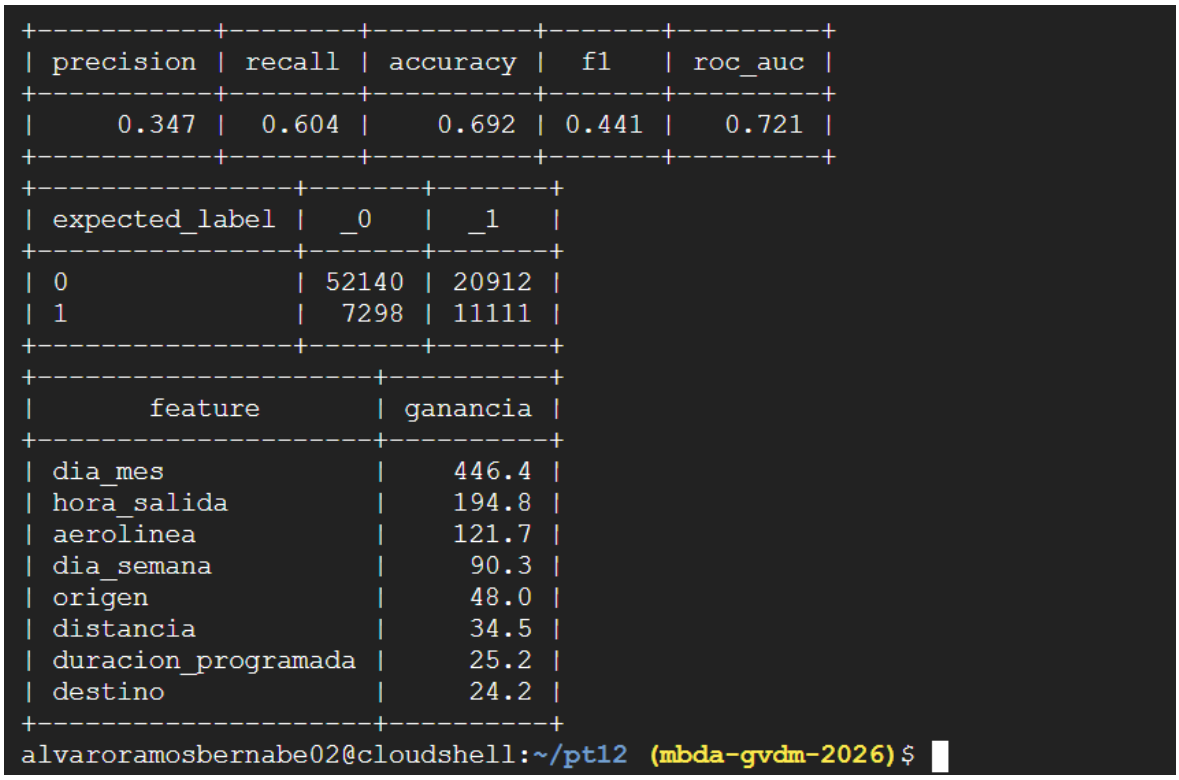


Figura 10. `ML.EVALUATE`, `ML.CONFUSION_MATRIX` y `ML.FEATURE_IMPORTANCE` del modelo entrenado en BigQuery ML.

Resultados: AUC 0,721, *recall* 0,604, precisión 0,347 y *accuracy* 0,692. La precisión baja es consecuencia directa de `auto_class_weights`: el modelo se vuelve sensible a la clase minoritaria y prefiere equivocarse por exceso (20.912 falsos positivos) antes que dejar escapar retrasos. Es la decisión correcta para un aviso operativo, y el umbral puede endurecerse después sobre la probabilidad guardada en la tabla de predicciones. La importancia de variables sitúa en cabeza el **día del mes** (446,4 de ganancia) —enero de 2015 incluye las tormentas invernales del noreste, que concentran los retrasos en días concretos—, seguido de la **hora de salida** (194,8), que refleja el efecto acumulativo a lo largo del día, y la **aerolínea** (121,7).

Frente al ejercicio 1 (AUC 0,777 con Random Forest), este modelo trabaja sobre un mes concreto y con menos variables derivadas, por lo que no son cifras directamente comparables; la ventaja aquí es de arquitectura: entrenar sobre **457.013 vuelos sin mover un solo byte fuera del almacén**.

5.4. Resultados y visualización

`ML.PREDICT` genera la tabla `predicciones` con la probabilidad de retraso de cada vuelo, y una agregación construye `kpi_aerolinea`, la capa de consumo del cuadro de mando.

```
FROM pt12.predicciones GROUP BY aerolinea ORDER BY pct_retraso_real DESC; -- at [7:1]
Created mbda-gvdm-2026.pt12.kpi_aerolinea

SELECT * FROM pt12.kpi_aerolinea; -- at [11:1]
+-----------+--------+------------------+----------------------+
| aerolinea | vuelos | pct_retraso_real | pct_retraso_previsto |
+-----------+--------+------------------+----------------------+
| MQ        |  27568 |             33.5 |                 53.3 |
| F9        |   6735 |             31.0 |                 51.0 |
| NK        |   8632 |             26.4 |                 52.7 |
| OO        |  46655 |             23.6 |                 51.9 |
| B6        |  20482 |             23.3 |                 47.5 |
| EV        |  48084 |             22.1 |                 49.9 |
| UA        |  37363 |             21.7 |                 49.5 |
| AA        |  43074 |             21.3 |                 47.1 |
| WN        |  98060 |             18.5 |                 43.5 |
| US        |  32478 |             17.6 |                 46.7 |
| VX        |   4647 |             15.9 |                 47.3 |
| AS        |  13151 |             14.1 |                 43.7 |
| HA        |   6408 |             13.2 |                 46.0 |
| DL        |  63676 |             12.2 |                 39.6 |
+-----------+--------+------------------+----------------------+
alvaroramosbernabe02@cloudshell:~/pt12 (mbda-gvdm-2026)$
```

Figura 11. Creación de las tablas de resultados y consulta de `pt12.kpi_aerolinea`.

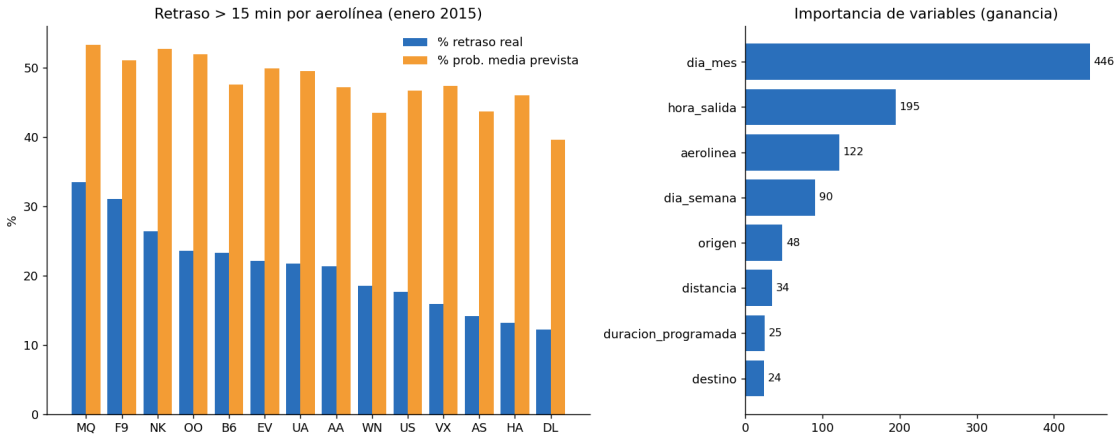


Figura 12. Cuadro de mando sobre la capa de resultados: retraso real frente a riesgo medio previsto por aerolínea e importancia de variables del modelo.

El cuadro de mando responde a la pregunta de negocio: **MQ (Envoy Air, 33,5 %), F9 (Frontier, 31,0 %) y NK (Spirit, 26,4 %)** encabezan el retraso real, frente a **DL (Delta, 12,2 %)** en el extremo opuesto, casi tres veces mejor. El riesgo medio previsto ordena a las compañías igual que el retraso observado, aunque desplazado hacia arriba por el reequilibrado de clases, de modo que sirve como **ranking** de riesgo y no como probabilidad calibrada. En producción, la tabla `kpi_aerolinea` se conecta a Looker Studio, que lee de BigQuery sin copiar datos y se refresca en cada ejecución del pipeline.

5.5. Equivalencia en AWS

La misma arquitectura se traslada servicio a servicio a AWS. El enunciado puntúa más alto esta opción; se documenta el diseño equivalente, aunque la implementación se ha realizado sobre Google Cloud:

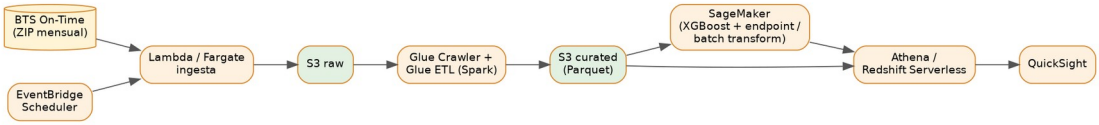


Figura 13. Arquitectura equivalente en AWS.

Etapa	Google Cloud (implementado)	AWS (equivalente)
Ingesta programada	Cloud Run Jobs + Cloud Scheduler	AWS Batch o Fargate Task + EventBridge Scheduler
Registro de imágenes	Artifact Registry	Amazon ECR
Lago de datos	Cloud Storage	Amazon S3 (zonas raw/curated/gold)
Catálogo y procesado	BigQuery (SQL)	AWS Glue Catalog + Glue ETL o Athena
Almacén analítico	BigQuery	Amazon Redshift Serverless o Athena sobre S3
ML	BigQuery ML	Amazon SageMaker (o Redshift ML con SQL)
Visualización	Looker Studio	Amazon QuickSight
Orquestación	Cloud Workflows / Composer	AWS Step Functions o MWAA

5.6. Eliminación de los servicios cloud

Conforme al apartado de facturación del enunciado, todos los recursos levantados se han eliminado al terminar: el *job* de Cloud Run, el bucket del lago con su contenido, el conjunto de datos de BigQuery (tablas y modelo) y el repositorio de imágenes creado por el despliegue desde código fuente.

```
gcloud run jobs delete ingesta-vuelos --region=$REG --quiet
gcloud storage rm --recursive gs://$LAGO --quiet
bq rm -r -f -d $PROY:pt12
gcloud artifacts repositories delete cloud-run-source-deploy --location=$REG --
quiet
```

```
Deleting [ingesta-vuelos]...done.
Deleted job [ingesta-vuelos].

Delete request issued for: [cloud-run-source-deploy]
Waiting for operation [projects/mbda-gvdm-2026/locations/europe-west1/operations/f5ce9b99-f643-49ce-8516-6689e271ca62] to complete...done.
Deleted repository [cloud-run-source-deploy].
# Recursos PT12 tras la limpieza
0
0
0
0
job, dataset, bucket y repositorio: 0 coincidencias
alvaroramosbernabe02@cloudshell:~/pt12 (mbda-gvdm-2026)$
```

Figura 14. Eliminación de los recursos y comprobación posterior: 0 coincidencias para el job, el conjunto de datos, el bucket y el repositorio de imágenes.

Conclusiones

- Con variables conocidas antes del despegue, el techo de predicción del retraso se sitúa en torno a **0,79 de AUC**: el Random Forest hiperparametrizado (0,7920) y la red densa (0,7907) quedan empatados en la práctica, y ambos superan con claridad a la regresión logística (0,7316).
- La validación cruzada apenas cambia la estimación puntual con 90.000 registros, pero es la que permite decidir si una diferencia entre modelos es significativa.

- El sobreajuste no se elimina, se administra: la mejor configuración conserva una brecha de 0,173 entre entrenamiento y test y aun así es la que mejor generaliza.
- La arquitectura cloud demuestra el mismo problema resuelto a escala real —457.013 vuelos— con servicios gestionados, entrenamiento en SQL dentro del propio almacén y coste proporcional al uso, con todos los recursos eliminados al finalizar.